ALEXANDRE Ewan, Groupe 1

18/10/2026

TP N°5 : SQL ET TRIGGERS

Table des matières

1) Problématique 1

2) Exercices 1

Exercice 1 : Parc immobilier 1

Exercice 2 : Des produits au prix variant au fil du temps 5

Exercice 3 : Location de véhicules 8

3) Conclusion 9

1) Problématique

Nous cherchons à concevoir et mettre en place des déclencheurs (triggers) sous MySQL afin d'automatiser des contrôles d'intégrité et synchroniser des valeurs calculées sur plusieurs modèles relationnels. Pour cela, nous allons voir comment utiliser les événements BEFORE et AFTER pour interdire des insertions ou modifications non conformes, et recalculer automatiquement des données métiers.

2) Exercices

Exercice 1 : Parc immobilier

On commence par créer la base de données et les tables nécessaires (Immeuble, Appartement, Photo, Piece) dans Notepad++ :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo1_tables.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

exo1_tables.sql

```sql
CREATE DATABASE IF NOT EXISTS tp5_exo1;
USE tp5_exo1;

CREATE TABLE Immeuble(
    Id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    adrNum VARCHAR(7) NOT NULL,
    adrVoie VARCHAR(100) NOT NULL,
    adrCodePostal VARCHAR(6) NOT NULL,
    adrVille VARCHAR(30) NOT NULL,
    fibreOptique TINYINT NOT NULL,
    parkingPrivatif TINYINT NOT NULL
    );

CREATE TABLE Appartement(
    immeuble INT NOT NULL,
    num INT NOT NULL,
    description LONGTEXT,
    loyer DOUBLE NOT NULL,
    superficie DOUBLE NOT NULL,
    terrasse TINYINT NOT NULL,
    classeConso CHAR(1) NOT NULL,
    chauffage CHAR(1) NOT NULL,
    placeParking TINYINT NOT NULL,
    prixParking DOUBLE,
    CONSTRAINT pk_appartement PRIMARY KEY (immeuble, num),
    CONSTRAINT fk_immeuble FOREIGN KEY (immeuble) REFERENCES Immeuble(Id)
    );

CREATE TABLE Photo(
    immeuble INT NOT NULL,
    appartement INT NOT NULL,
    reference INT NOT NULL,
    titre VARCHAR(75),
    description VARCHAR(255),
    uri VARCHAR(120) NOT NULL,
    CONSTRAINT pk_photo PRIMARY KEY (immeuble, appartement, reference),
    CONSTRAINT fk_appartement_photo FOREIGN KEY (immeuble, appartement) REFERENCES Appartement(immeuble, num)
    );
```

Question 1 : Trigger de contrôle du prix de parking

On rédige le trigger BEFORE INSERT sur la table Appartement pour vérifier que le prix de parking reste NULL si l'appartement ne possède pas de place :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo1_trigger_q1.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

exo1_trigger_q1.sql

```sql
USE tp5_exo1;

DELIMITER |
CREATE TRIGGER verif_parking_insert BEFORE INSERT ON Appartement
FOR EACH ROW
BEGIN
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erreur : le prix de la place de parking doit etre NULL sans parking';
    END IF;
END |
DELIMITER ;
```

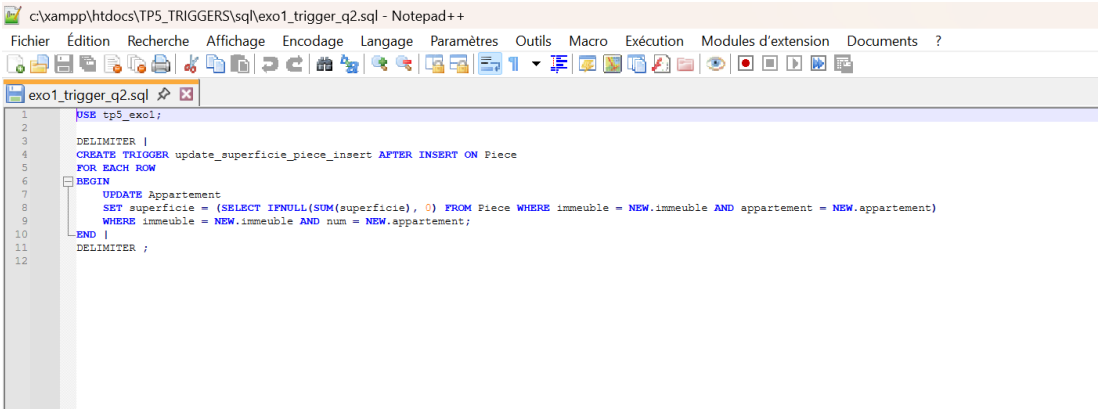
On teste ensuite le trigger dans le terminal MySQL avec une insertion valide et une insertion invalide :

MySQL_Exo1_Test_Q1 - c:\xam

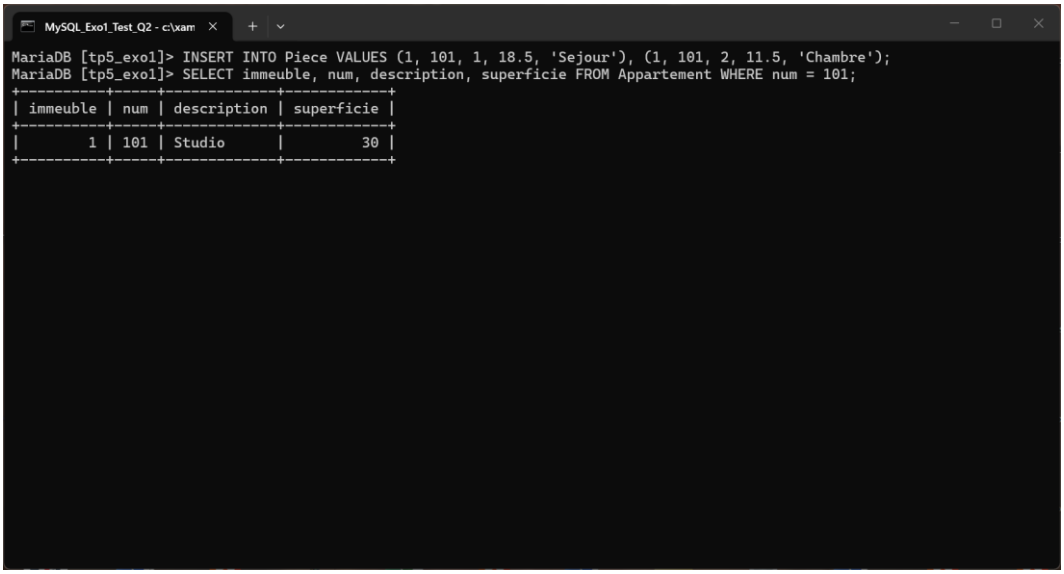
```
MariaDB [tp5_exo1]> INSERT INTO Immeuble VALUES (1, '12', 'Rue Victor Hugo', '75001', 'Paris', 1, 1);
MariaDB [tp5_exo1]> INSERT INTO Appartement VALUES (1, 101, 'Studio', 650, 30, 0, 'C', 'E', 0, NULL);
MariaDB [tp5_exo1]> INSERT INTO Appartement VALUES (1, 102, 'T2', 850, 50, 0, 'C', 'E', 0, 75.0);
ERROR 1644 (45000) at line 1: Erreur : le prix de la place de parking doit etre NULL sans parking
```

Question 2 : Calcul automatique de la superficie de l'appartement à l'insertion d'une pièce

On crée le trigger AFTER INSERT sur la table Piece pour recalculer la superficie totale de l'appartement à chaque nouvelle pièce ajoutée :



On teste l'ajout de pièces dans le terminal MySQL et on vérifie la mise à jour de la superficie de l'appartement :



Question 3 : Adaptation des contraintes d'appartement

On adapte le trigger d'insertion pour vérifier que l'immeuble dispose bien d'un parking privatif, initialiser la superficie à 0, et on crée un trigger BEFORE UPDATE interdisant la modification directe de la superficie :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo1_trigger_q3.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

exo1_trigger_q3.sql

```sql
USE tp5_exo1;

DROP TRIGGER IF EXISTS verif_parking_insert;

DELIMITER |
CREATE TRIGGER verif_appartement_insert BEFORE INSERT ON Appartement
FOR EACH ROW
BEGIN
    DECLARE has_parking TINYINT;

    SELECT parkingPrivatif INTO has_parking FROM Immeuble WHERE id = NEW.immeuble;

    IF has_parking = 0 AND NEW.placeParking = 1 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Impossible d avoir une place sans parking privatif dans l immeuble';
    END IF;

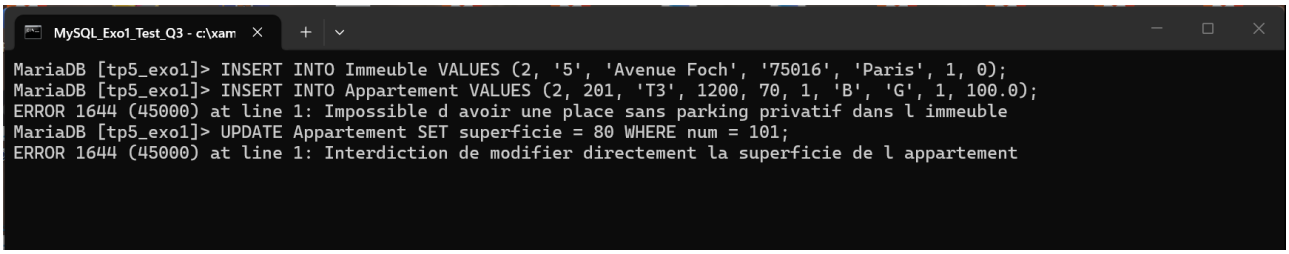
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Le prix du parking doit etre NULL sans place de parking';
    END IF;

    SET NEW.superficie = 0;
END |

CREATE TRIGGER verif_appartement_update BEFORE UPDATE ON Appartement
FOR EACH ROW
BEGIN
    DECLARE total_pieces DOUBLE;
    SELECT IFNULL(SUM(superficie), 0) INTO total_pieces
    FROM Piece
    WHERE immeuble = NEW.immeuble AND appartement = NEW.num;

    IF NEW.superficie != total_pieces THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Interdiction de modifier directement la superficie de l appartement';
    END IF;
END |
DELIMITER ;
```

On teste ces nouvelles contraintes dans le terminal MySQL :



Question 4 : Mise à jour et suppression de pièces

On écrit les triggers AFTER UPDATE et AFTER DELETE sur la table Piece afin que toute modification ou suppression d'une pièce recalcule immédiatement la superficie de l'appartement :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo1_trigger_q4.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

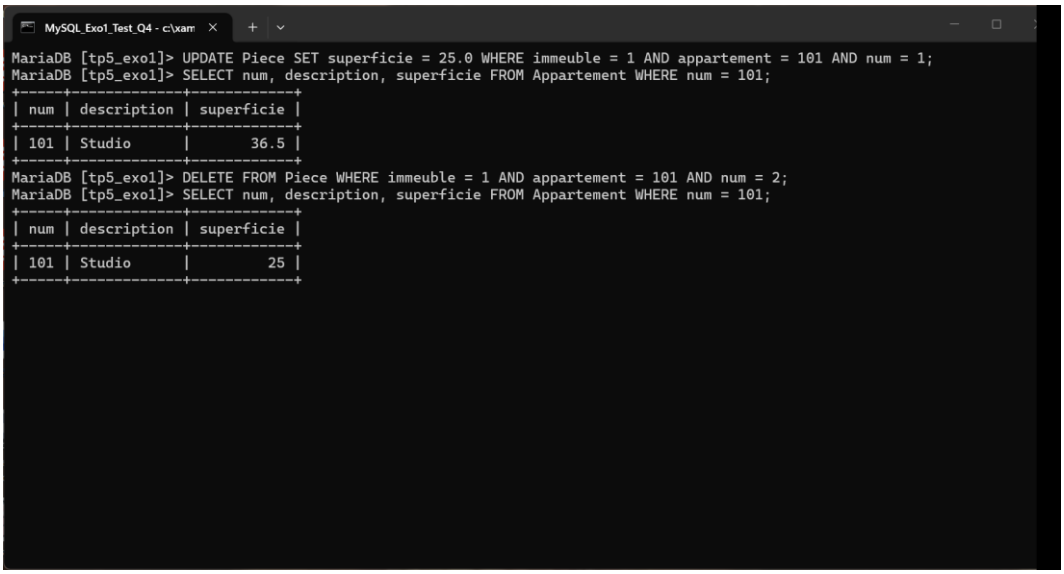
exo1_trigger_q4.sql

```sql
USE tp5_exo1;

DELIMITER |
CREATE TRIGGER update_superficie_piece_update AFTER UPDATE ON Piece
FOR EACH ROW
BEGIN
    UPDATE Appartement
    SET superficie = (SELECT IFNULL(SUM(superficie), 0) FROM Piece WHERE immeuble = NEW.immeuble AND appartement = NEW.appartement)
    WHERE immeuble = NEW.immeuble AND num = NEW.appartement;
END |

CREATE TRIGGER update_superficie_piece_delete AFTER DELETE ON Piece
FOR EACH ROW
BEGIN
    UPDATE Appartement
    SET superficie = (SELECT IFNULL(SUM(superficie), 0) FROM Piece WHERE immeuble = OLD.immeuble AND appartement = OLD.appartement)
    WHERE immeuble = OLD.immeuble AND num = OLD.appartement;
END |
DELIMITER ;
```

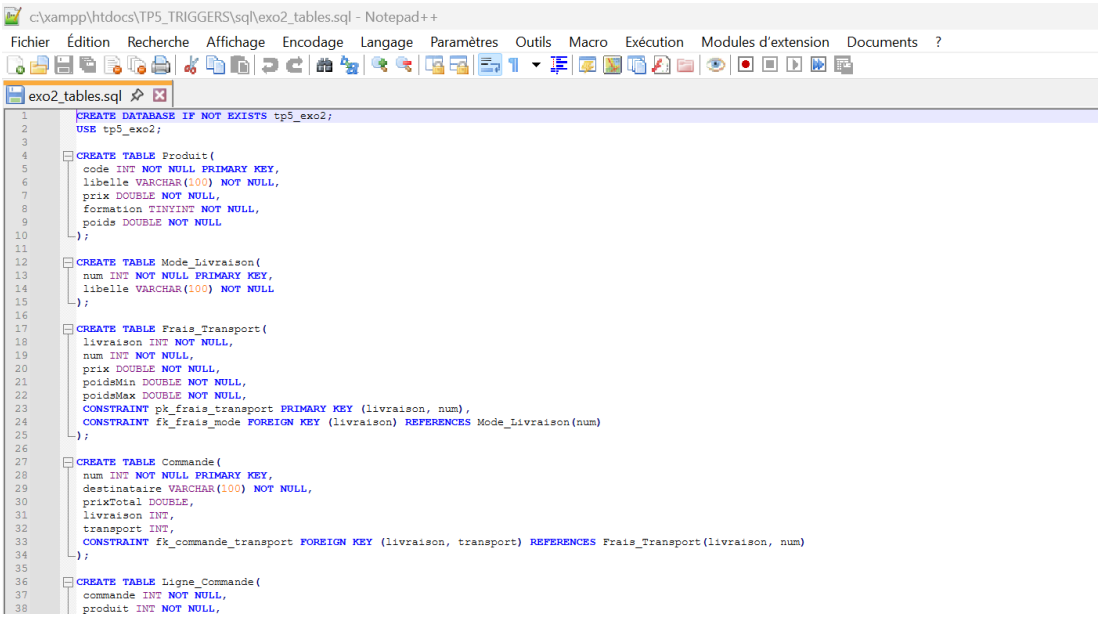
On vérifie le recalcul lors de la modification puis de la suppression d'une pièce dans la console MySQL :



Exercice 2 : Des produits au prix variant au fil du temps

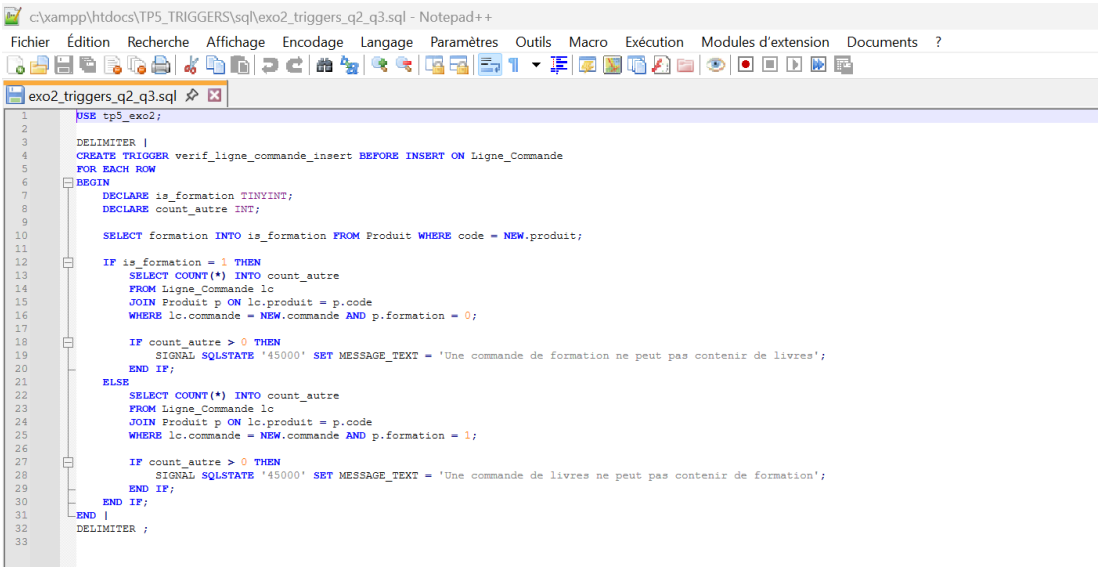
Question 1 : Script de création des tables

On crée le schéma relationnel complet sous Notepad++ (tables Produit, Mode_Livraison, Frais_Transport, Commande et Ligne_Commande) :

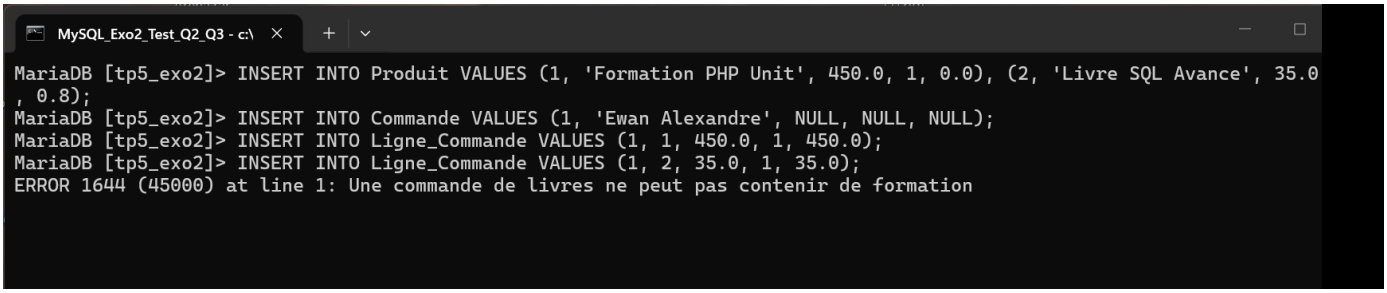


Questions 2 et 3 : Exclusion mutuelle entre formation et livres

On rédige le trigger BEFORE INSERT sur Ligne_Commande pour interdire qu'une commande mélange une formation et un ou plusieurs livres :



On teste la tentative d'ajout d'un livre dans une commande contenant déjà une formation dans le terminal MySQL :



Questions 4 et 5 : Frais de transport et mise à jour automatique du prix total

On adapte le trigger pour interdire les frais de transport sur les commandes de formation, et on ajoute un trigger AFTER INSERT sur Ligne_Commande qui recalcule automatiquement le prix total de la commande :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo2_triggers_q4_q5.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

exo2_triggers_q4_q5.sql

```sql
USE tp5_exo2;

DROP TRIGGER IF EXISTS verif_ligne_commande_insert;

DELIMITER |
CREATE TRIGGER verif_ligne_commande_insert BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    DECLARE is_formation TINYINT;
    DECLARE count_livres INT;
    DECLARE has_livraison INT;

    SELECT formation INTO is_formation FROM Produit WHERE code = NEW.produit;

    IF is_formation = 1 THEN
        SELECT COUNT(*) INTO count_livres
        FROM Ligne_Commande lc
        JOIN Produit p ON lc.produit = p.code
        WHERE lc.commande = NEW.commande AND p.formation = 0;

        IF count_livres > 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Une commande de formation ne peut pas contenir de livres';
        END IF;

        SELECT livraison INTO has_livraison FROM Commande WHERE num = NEW.commande;
        IF has_livraison IS NOT NULL THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Une commande de formation ne peut pas comporter de frais de transport';
        END IF;
    ELSE
        SELECT COUNT(*) INTO count_livres
        FROM Ligne_Commande lc
        JOIN Produit p ON lc.produit = p.code
        WHERE lc.commande = NEW.commande AND p.formation = 1;

        IF count_livres > 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Une commande de livres ne peut pas contenir de formation';
        END IF;
    END IF;
```

On teste la mise à jour automatique du montant total avec les frais de transport dans la console MySQL :

MySQL_Exo2_Test_Q4_Q5 - c:\

```
MariaDB [tp5_exo2]> INSERT INTO Mode_Livraison VALUES (1, 'Colissimo');
MariaDB [tp5_exo2]> INSERT INTO Frais_Transport VALUES (1, 1, 8.5, 0.0, 2.0);
MariaDB [tp5_exo2]> INSERT INTO Commande VALUES (2, 'Client Livres', NULL, 1, 1);
MariaDB [tp5_exo2]> INSERT INTO Ligne_Commande VALUES (2, 2, 35.0, 2, 70.0);
MariaDB [tp5_exo2]> SELECT num, destinataire, prixTotal FROM Commande WHERE num = 2;
+-----+---------------+-----------+
| num | destinataire  | prixTotal |
+-----+---------------+-----------+
|   2 | Client Livres |      78.5 |
+-----+---------------+-----------+
```

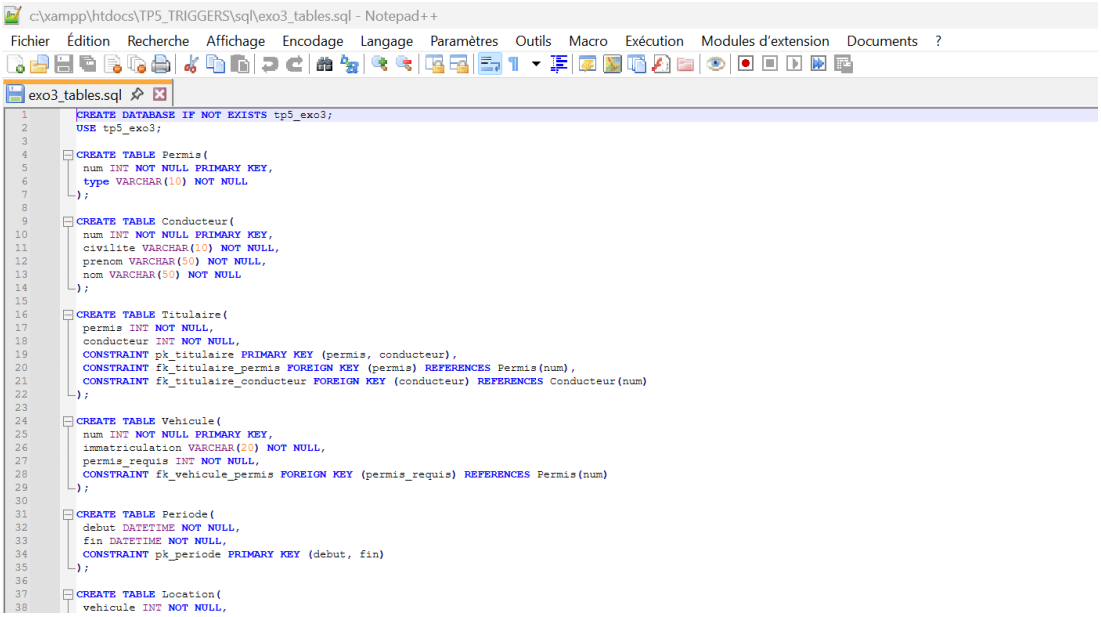
Question 6 : Cas non traités par les triggers

Au travers des questions 1 à 5, plusieurs cas de gestion réels n'ont pas été traités :
- Les modifications (UPDATE) sur la table Ligne_Commande : si on modifie la quantité ou le produit d'une ligne existante, le prix total de la commande ne sera pas recalculé.
- Les suppressions (DELETE) d'une ligne de commande : si un article est annulé, le prix total ne diminue pas automatiquement.
- La modification ultérieure des frais de transport dans Commande : un client pourrait ajouter un mode de livraison payant après avoir créé une commande de formation.
- L'évolution du prix d'un produit dans la table Produit : si le tarif unitaire change, les commandes passées doivent conserver leur prix historique et ne pas être impactées rétroactivement.

Exercice 3 : Location de véhicules

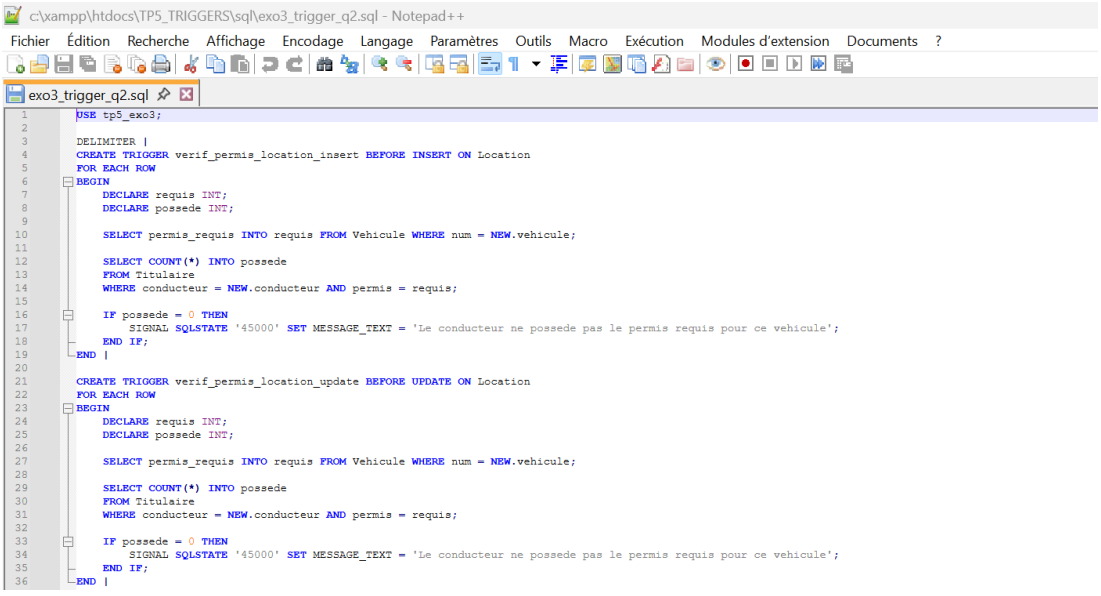
Question 1 : Script de création des tables

On rédige le script de création des tables pour la gestion du parc de véhicules (Permis, Conducteur, Titulaire, Véhicule, Période, Location) sous Notepad++ :

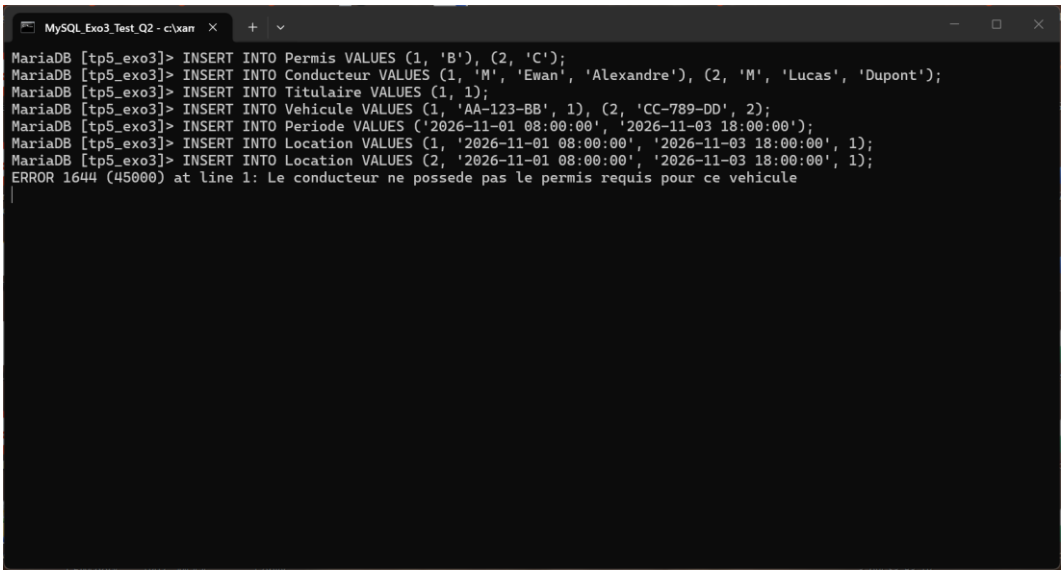


Question 2 : Trigger de vérification du permis requis pour la location

On crée les triggers BEFORE INSERT et BEFORE UPDATE sur la table Location afin de contrôler que le conducteur possède bien le permis exigé par le véhicule :



On teste la location dans le terminal MySQL avec un conducteur habilité, puis avec un véhicule exigeant un permis qu'il ne détient pas :



3) Conclusion

En conclusion, ce TP m'a permis d'approfondir la mise en oeuvre des déclencheurs SQL sous MySQL. J'ai pu vérifier concrètement l'intérêt des événements BEFORE pour bloquer les données invalides à la source et des événements AFTER pour synchroniser automatiquement les calculs (superficie d'un appartement, prix total d'une commande) sans faire reposer cette logique sur l'application.